The Execution Graph Contract (frozen)

> **Status:** FROZEN as of Sprint 2D (pre-2D.2). Changes to the section model below require a design review and a schema-version bump — not an ad-hoc field added to a node.
>
> **Scope:** This document freezes the canonical shape of `ScenarioNode` / `ScenarioGraph` (`src/graph/scenario-graph.ts`) before Sprint 2D adds executable Actions (2D.3) and executable Assertions (2D.4). It exists so that six months from now nobody drops `resolvedDataset` inside `semantics`, or an assertion inside `metadata`.

---

1. Why this is now called the Execution Graph

The type is still named `ScenarioGraph` in code (renaming is deferred to avoid churn), but the **mental model has changed**. Through Sprint 2C it described what scenarios exist. After Sprint 2D it will describe exactly how each scenario executes:

```
Requirement
   ↓
Scenario Graph ──────────────► Execution Graph
   (what scenarios exist)        (how each scenario executes)
                                   • execution data   (2D.2)
                                   • executable actions   (2D.3)
                                   • executable assertions (2D.4)
```

Once complete, it is the **core domain model of LevelUp AI** — the single canonical execution model consumed by:

```
Script Generation · Healing · Replay · Self-Healing · Impact Analysis · Coverage Gap Analysis
```

Every one of those reads the same node. None re-derives it.

---

2. The canonical `ScenarioNode` — seven sections

A node is organised into **seven ownership sections**. Every field lives in exactly one section, chosen by the placement rules in §3. This is the frozen contract:

```
ScenarioNode {

  // ── 1. IDENTITY ──────────────────────────────  immutable, defines "which scenario"
  id                 // stable KB scenarioId — the join key for every consumer
  title
  objective

  // ── 2. KNOWLEDGE (semantics) ─────────────────  immutable, "what the scenario
MEANS"
  semantics {
    variableUnderTest
    preconditions
    variation
    expectedBehavior
    requiredDataRole          // a data ROLE ("a registered user") — never a resolved
dataset
  }

  // ── 3. RESOURCES  (reserved) ─────────────────  immutable, "what the scenario
NEEDS"
  resources {
    dataRoles                 // e.g. ["registered_user"]  (the role requirement)
    services                  // e.g. ["inventory_api"]     (required backing
services / mocks)
    environment               // required env class (e.g. "staging-capable")
    browser                   // required browser capability (e.g. "chromium")
    locale                    // required locale (e.g. "en-US")
    // featureFlags / device / tenant / oauthClient — natural future members
  }

  // ── 4. EXECUTION ──────────────────────────────  runtime, "what THIS run actually
USED"
  execution {
    resolvedDataset           // 2D.2 — concrete record resolved from re-
sources.dataRoles
    // resolvedBrowser / resolvedLocale / resolvedEnvironment — natural future members
  }

  // ── 5. ACTIONS  (added in 2D.3) ──────────────  immutable, "the executable steps"
  actions[] {
    stepId
    action                    // navigate | fill | click | select | check | upload |
wait | verify
    target                    // stable element identity (semantic key), NOT a raw
locator
    value                     // literal, or @dataset.* reference resolved from execu-
tion
  }

  // ── 6. ASSERTIONS  (added in 2D.4) ───────────  immutable, "the executable expected
outcomes"
  assertions[] {
    type                      // url | visible | hidden | text | value | enabled |
disabled | count | error
    target                    // stable element identity (when the assertion is
element-scoped)
    value                     // expected literal / pattern
  }

  // ── 7. QA METADATA + PROVENANCE ──────────────  diagnostic / classification
  coverageType, priority, severity, riskArea, tags,
  automationReady, automationComplexity, selectorAvailability,
```

```
  source, sourceEvidence, grounded,
  expectedResults            // human-readable outcomes (superseded for execution by
§6)
  dependencies               // typed edges live on the graph; per-node refs here if
needed
  metadata                   // confidence, timings, telemetry — never behaviour-bear-
ing
}
```

> **Note on reserved slots (`resources`, `actions[]`, `assertions[]`):** these are **reserved, not yet in code**. The node's existing invariant ("every field must serve ≥2 consumers") means each is added in the PR that also adds its first consumer — `resources` when a second consumer beyond the Dataset Resolver needs it (env/browser/locale requirements), `actions[]` in 2D.3, `assertions[]` in 2D.4. Today the data ROLE requirement still lives in `semantics.requiredDataRole`; it migrates to `resources.dataRoles` when the `resources` section lands. This document freezes where each will go and what it will contain so those PRs are pure fills, not redesigns.

---

3. Placement rules — what belongs in each section

Section	Immutable?	Answers…	Example	Do NOT put here
identity	✅ immutable	Which scenario is this?	`id: "auth-neg-wrong-password"`	anything run-varying
semantics	✅ immutable	What does it fundamentally mean?	`variableUnderTest: "password"`	resolved values, locators, requirements
resources	✅ immutable	What does it NEED to run?	`dataRoles: ["registered_user"]`	resolved/selected values (those are execution)
execution	❌ runtime	What did THIS run actually use?	`resolvedDataset: {username,…}`	anything that changes identity
actions	✅ immutable	What steps execute, in order?	`{action:"fill", target:"username"}`	resolved values inline (use `@dataset.*`)
assertions	✅ immutable	What outcomes are verified?	`{type:"url", value:"/inventory"}`	prose like “login succeeds”
metadata	❌ diagnostic	How confident / how measured?	`confidence: 0.82`	anything behaviour-bearing

The three-question separation — the reason `resources` and `execution` are distinct sections:

Question	Section	Example
What does this scenario mean?	`semantics`	variable under test = password
What does this scenario need?	`resources`	a registered user, chromium, en-US
What did this run actually use?	`execution`	standard_user, Chrome 139, en-US

`resources` is the immutable requirement (“I need a registered user”); `execution` is the mutable resolution (“this run used standard_user”). They are different abstractions and must never be merged. This is exactly why `requiredDataRole` (the need) and `resolvedDataset` (the resolution) cannot share

a
section.

The decision test (apply to every new field)

1. **Does it change if the same scenario runs with a different dataset / env / browser?**
   → YES → is it the requirement (immutable need) or the resolved value (this run)?
   → requirement → `resources`; resolved value → `execution`. NO → continue.
2. **Is it a value the run produces or measures (never an input)?**
   → YES → `metadata`. NO → continue.
3. **Is it a step the browser performs?** → `actions`.
4. **Is it an outcome the test verifies?** → `assertions`.
5. **Does it define what the scenario means independent of any run?** → `semantics`.
6. **Does it identify which scenario this is?** → `identity`.

If a field seems to fit two sections, it is probably two fields. Split it. (The canonical example: `requiredDataRole` / `dataRoles` is a `resources` requirement; the record it resolves to is `execution`.)

Hard invariants (still enforced)

- **A ROLE requirement is never a resolved dataset.** The data-role requirement (`semantics.requiredDataRole`
  today, `resources.dataRoles` once `resources` lands) is an immutable NEED. The Dataset Resolver maps role
  → record; the resolved record lands in `execution.resolvedDataset`. A resolved record must **never** appear
  in `semantics` or `resources` — requirement and resolution are different abstractions.
- **`target` is a semantic element identity, never a raw locator string.** Locator resolution is Script
  Gen's job at emit time; the graph stays framework-neutral.
- **`value` may be a `@dataset.*` reference.** Actions reference execution data symbolically so the same
  action list is reusable across datasets — the value is bound from `execution.resolvedDataset` at emit time.
- **Every shared-node field serves ≥2 consumers.** Single-consumer data belongs in that consumer, not on
  the node. The graph is a contract, not a junk drawer.
- **Identity / semantics / resources / actions / assertions are immutable per scenario.** Only `execution`
  and `metadata` vary run-to-run. The fingerprint hashes identity + semantics + resources + actions +
  assertions — never execution or metadata.

---

4. Schema-version governance

`SCENARIO_GRAPH_SCHEMA_VERSION` (currently `'1.0.0'`) is the contract version. Bump it when the shape changes:

- **PATCH** — additive optional field within an existing section, backward-compatible.

- **MINOR** — new section slot populated for the first time (`resources` , 2D.3 `actions` , 2D.4 `assertions`).
- **MAJOR** — a field moves sections, is removed, or changes meaning (requires migration + review).

Persisted graphs record the version they were built with; readers must tolerate older optional-field-absent
graphs (as they already do for `semantics` and `execution`).

---

5. Migration state (Sprint 2D)

Section	Present in code today	Populated by	First consumer
identity	✅	builder	all
semantics	✅ (optional)	builder ← KB (`getScenarioSemantics`)	Test Case Lab, Script Gen, Healing, Dataset Resolver
resources	🔲 reserved (role req. in `semantics.requiredDataRole` today)	builder ← KB	Dataset Resolver (+ future env/browser consumers)
execution	✅ (optional; `resolvedDataset`)	builder ← Dataset Resolver	Test Case Lab, Script Gen (2D.2)
actions	🔲 reserved	builder (2D.3)	Script Gen (2D.3)
assertions	🔲 reserved	builder (2D.4)	Script Gen (2D.4)
metadata	✅ (scattered)	builder / validator	RTM, telemetry

The rule for the rest of Sprint 2D: additions land in the reserved slots above, in the PR that also adds their consumer. No field is added anywhere else on the node without updating this document and bumping
the schema version.

---

6. Definition of “frozen”

The contract is frozen means:

1. The seven sections and their ownership are fixed. New data goes into an existing section per §3, or is
   not a node concern.
2. `resources` , `actions[]` and `assertions[]` have a **known, documented shape** before they are implemented — so those PRs populate a pre-agreed slot rather than debating structure mid-sprint.

3. Any deviation is a reviewed schema change, not a silent field.

This is what lets every subsequent sprint be additive: the destination for each new capability is already
decided. **No more structural discussions — only fill the reserved sections.**

Next step: Sprint 2D.2 — populate & consume `execution.resolvedDataset` (additive; no inference removed).